Activity 1, Advanced - Intro to Spatial Mapping

Activity 1, Advanced - Intro to Spatial Mapping

Download this activity

In this adventure, students will begin mapping the SVG Canvas units to physical and tangible drawing surfaces utilizing tactile drawing paper. Students will draw basic shapes on tactile dot or graph paper and understand how to mark and understand shape positions around the canvas and measure the dimensions of their shapes.

Setup - Activity 1

This activity can be broken down into several elements for lesson planning.

Objectives

- Learn how a drawing on paper can be mapped into digital units by presenting a tactile coordinate system.
- Teaching that the origin is the top-left corner of any drawing in either portrait or landscape orientation and is represented as (0,0) or x=- and y=0.
- Teaching that we can think of the relative units as 100 units per inch, and that each dot on the provided dot paper or each line in the graph paper represents 50 unit increments across the paper.
- Have fun creating designs with basic shapes and understanding how to feel and note their dimensions/sizes using the tactile markings on the paper.

Materials Needed

- APH Draftsman Tactile Drawing Board or APH TactileDoodle
- Tactile Dot or Graph paper (Provided in TADA materials)
- Stylus or Pen
- Tactile Rulers
  - Cardstock or Cardboard/ for a straight edge that can be folded

Alternate Materials

- Sensational Blackboard

Ideas for Carrying Out This Activity

- Use basic shapes at first, offer up suggestions like making a circle 150 units in diameter, squares and rectangles of various sizes,and try connecting various dots together with lines to make triangles and polygons.
- If a student wants to progress further than basic shapes, encourage them to build a scene using various shapes and straight lines.

Adventure Map: Activity 1

Teaching tip: Provide sufficient time for the student to explore, develop skills, and have fun at each step! Encouraging creativity and personal preferences for drawing as much as possible. Some students may be able to accomplish each step in one session; most students will need several sessions to complete the adventure.

1. Introduction

By this point through TADA, students should be comfortable drawing basic shapes via tactile drawing surfaces. Ask your students how they think computers know how large or small to render graphics and where to place them on the screen or paper.

The answer is numbers, specifically the numbers the students give the computer. When drawing shapes, ideas like size, positioning, line thickness, and more are understood by the computer as "attributes" and the numbers that change these attributes are called "values." We have to know the values of the various attributes of the shapes we draw in order to accurately input them on the computer.

2. Create Your Graphic

Provide your student with drawing materials. Ask your student to draw a series of basic shapes with different attribute values on the tactile paper. The following are some example instructions:

- "Draw a circle that is 150 units in diameter in the center of the paper"
- "draw a rectangle 200 units wide and 50 units tall, starting the top-left corner of the rectangle 100 units over on X and 300 units down on Y,"
- "Draw a right triangle with the top point starting at 50 units over on X and 50 units down on Y, the second point 50 units over on X and 200 units down on Y, and the third point 250 units over on X and 200 units down on Y, and connect all the dots together."

Once students get an understanding of how to think about the attributes and the values as they appear on the tactile paper, have them explore drawing their

own scene or set of shapes, sticking to basic shapes and straight lines.

3. Talking through Design Process

- Have students speak out their shape attributes and values.
- Have students toss out shapes and values to the other students to try drawing on their own workspaces.
- Tie this back to the previous lessons about communicating drawing instructions.

4. Conclusion

Once students are comfortable with the canvas space, get them excited for the next step of translating their designs into actual code on the computer that will render it digitally.

- Does feeling their drawings and knowing the dimensions make them feel more assured about the numbers they'll be using to build their shapes?
- Does the canvas system make sense?
- Reinforce that the top-left corner of the drawing is (0, 0) and is known as the origin, and that for a standard 8.5"x11" sheet of Letter paper in landscape orientation, this means that the top-right corner can be represented as (1100, 0), the bottom-right corner as (1100, 850), and the direct center of the page as (550, 425).

Download the whole thing · **Next:** Activity 2: Digitizing Drawings with SVG Code · **Back to:** Adventure Map

Activity 2, Advanced - Digitizing Drawings with SVG Code

Activity 2, Advanced - Digitizing Drawings with SVG Code

Download this activity

In this activity, students will get their hands on SVG code and learn how to render their physical Activity 1 drawings as digital illustrations. Students will follow the syntax available on BlindSVG.com to build their basic shapes using the attributes and values from the first activity, then get visual and tactile confirmation of their end result, getting to compare the output to their original drawings.

This activity will involve computer use, text editor navigation and text editing/input, being able to open the finished SVG files in Google Chrome or Mozilla Firefox, and outputting via any tactile method.

Setup - Activity 2

This activity can be broken down into several elements for lesson planning.

Objectives

- Begin to understand how to write SVG code and map physical drawings to the digital canvas space.
- Learn syntax, code and shape ordering, file setup, and output.
- Learn how to communicate with a visual interpreter when assessing accuracy of the digital drawing, and feel differences between tactile/embossed output and the original tactile drawings.
- Enjoy creating their first digital illustration from drawn concept to final digital result!

Materials Needed

- APH Draftsman Tactile Drawing Board or APH TactileDoodle
- Tactile Dot or Graph paper (Provided in TADA materials)

- Stylus or Pen
- Tactile Rulers
  - Cardstock or Cardboard/ for a straight edge that can be folded
- Laptop or Desktop computer
- Text Editor: TextEdit or Text Mate on Mac, Notepadd++ on Windows
- Google Chrome or Mozilla Firefox (Safari may not work for this process)
- Tactile Graphics output method: embosser, Swell-form, visual tracing method on standard printer output

Alternate Materials

- Sensational Blackboard

Ideas for Carrying Out This Activity

- Read through and have students read through BlindSVG.com, specifically the Home Page, Setup page, and the Basic Shapes page.
- Pay attention to good typing and syntax usage. Be ready to hunt through code to debug any missing spaces, quotation marks, or incorrect values.
- Use Swell-form if available for final output.
- If no other tactile output methods are available, print drawings out on basic printers and set print setting to flip the drawing horizontally so it prints out a mirror image. Place this print on a Draftsman and trace the resulting image with a pen for the student, embossing the image and making it appear exactly as intended when the paper is flipped around to expose the embossed lines for the student.

Adventure Map: Activity 2

Teaching tip: Provide sufficient time for the student to explore, develop skills, and have fun at each step! Encouraging creativity and personal preferences for drawing as much as possible. Some students may be able to accomplish each step in one session; most students will need several sessions to complete the adventure. This is a very technical part of the adventure, and this may become frustrating depending on the technical aptitude of the students and their screen reader/magnification skills. Please give them enough time and space to solve for technical bugs and typing errors; Google Chrome will do a basic debug of SVG code and tell the student what lines have errors when opening the SVG within the browser.

1. Introduction

Students will now get their first chance to translate their tactile physical drawing into an actual digital illustration using SVG code. This requires a good understanding of the attributes and values of their drawn shapes and comfort with manipulating and editing text in a basic text editor.

2. Read BlindSVG.com

To get prepared for this task, have students read through the following pages of BlindSVG.com:

- BlindSVG Home Page
- BlindSVG Setup Page
- BlindSVG Shapes Page

This will give them information about SVG coding, how to set up their first SVG text files, explain the overall concept of the SVG units (reinforced by Adventure 1), and give them syntax to create basic shapes.

3. Build Template SVG Files

- Have students open a new text file, then have them copy and paste the SVG file template code from the BlindSVG setup page.
- In the opening SVG tag, if the student has a landscape drawing, ensure they set the width attribute to 1100 and the height attribute to 850. Swap these numbers for a portrait orientation.
- Have the students save their files with the ".svg" extension instead of ".txt."

4. Identify Shapes and Start Coding!

- From the BlindSVG Basic Shapes page, have students identify the basic shapes they've drawn and match them to the shapes available on the website. Have them list out all the shapes they need, and proceed to have them copy and paste the correct shape code from the site into their text files.
- The students should match the attributes and values they came up with from Adventure 1 and input them into the shape code in their files. Students can copy and paste or can type out the syntax directly, whatever is most comfortable for them.
- Students should be taught that the order of the code matters. The last line of code before the ending `</svg>` tag will be the top-most shape layer in the drawing.
- **Thought experiment:** consider each line of shape code as the same shape cut out in construction paper. Each line of code they write is telling the computer to lay down another shape of construction paper on top of the last one. Pieces of the drawing that the students want in the background or behind other layers should be written first in the code, and everything in the foreground or appearing in front of other objects should come last in the code.

5. First Output and Debugging

- Once the students have all entered their shapes and verify that their attributes and values are all correct, have them open their SVG file in

Chrome or Firefox.

- Did Chrome print any errors on the screen? If so, identify the line the error has occurred on and have the student go back to their file and find and fix the error. This can be a missing or errant space, missing quotation mark, missing angle bracket, a missing slash to close out a shape, or a bad value/broken syntax.
- If the drawing renders, celebrate with the student!

6. Output and Tactile Assessment

- Send the rendered drawings to an embosser or use the flipped print and trace method to turn the digital file into a tangible tactile adventure for the student to explore. Have them compare their output to their original drawings:
  - Is anything out of place?
  - Does anything need tweaking?
  - Inspired to add more and iterate on the drawing?

7. Final tweaks

- Basic tactile output requires black outlines, white negative space, and can contain one or two additional colors for embossers that can produce different dot heights. These fill colors seem to work best with shades of gray or the CSS Extended Color Keyword "oldlace" or "seashell."
- Swell form will produce cleaner lines along with a visual print on top of the tactile print, so this can be used as a final output mechanism if available.
- Students can go back in and play with their code to tweak and iterate their final results. Give them visual interpretation now that they have a tangible piece of artwork to work from based on their original numbers.
- Do they have ideas of what more they could create now that they've built their first digital graphic?
- Does the syntax flow for them? Is it hard to understand or grasp?
- What would make it easier for them to understand the process?
- Does knowing that blind designers are actively using this process to make logos, art, technical drawings, layouts, engineering plans, circuit diagrams, charts and graphs, and more get them excited about the possibilities of this process?

Previous: Activity 1: Intro to Spatial Mapping · Download the whole thing · **Next:** Activity 3: Iterating on SVG Drawings & Beyond · **Back to:** Adventure Map

Activity 3, Advanced - Iterating on SVG Drawings & Beyond

Activity 3, Advanced - Iterating on SVG Drawings & Beyond

Download this activity

In this final adventure of TADA, students will practice iterating their digital drawings by changing attributes of shapes to alter their appearance, moving them around, fixing problems, and getting comfortable enough with manipulating the SVG syntax to prepare for more advanced coding and drawing skills.

This adventure will involve computer use, text editor navigation and text editing/input, opening and navigating through their SVG files and altering the code, being able to open the finished SVG files in Google Chrome or Mozilla Firefox, and outputting via any tactile method.

These tasks should be able to be repeated for each change to the SVG file code and the previous embossed or tactile output retained to compare and contrast the differences between the previous drawing output and the current iteration. Students should be able to discern how the numbers and changes they've made are affecting the output and should be able to enjoy experimenting with subtle and big changes to help spatially map the code to what they are feeling with the tactile output.

Setup - Activity 3

This activity can be broken down into several elements for lesson planning.

Objectives

- Begin to spatially and mentally map how changing numbers and attribute values in the shapes can alter the tactile output.
- Create quick hypotheses about where shapes will appear on the paper when they input certain values, and assess accuracy with tactile output.
- Progress on verbal description of what they've changed and get more comfortable working with a visual interpreter on gauging the accuracy of what

they've changed and how the appearance of their drawing has been altered.
- Be ready to advance to learning paths, curves, and additional illustration skills both as they progress through BlindSVG.com and create more of their own graphics.

Materials Needed

- Laptop or Desktop computer
- Text Editor: TextEdit or Text Mate on Mac, Notepadd++ on Windows
- Google Chrome or Mozilla Firefox (Safari may not work for this process)
- Tactile Graphics output method: embosser, Swell-form, visual tracing method on standard printer output

Ideas for Carrying Out This Activity

- Read through and have students read through BlindSVG.com, specifically the Home Page, Setup page, and the Basic Shapes page.
- Pay attention to good typing and syntax usage. Be ready to hunt through code to debug any missing spaces, quotation marks, or incorrect values.
- Be prepared for multiple iterations of tactile output as students change the values in their code.
- Use Swell-form if available for final output.
- If no other tactile output methods are available, print drawings out on basic printers and set print setting to flip the drawing horizontally so it prints out a mirror image. Place this print on a Draftsman and trace the resulting image with a pen for the student, embossing the image and making it appear exactly as intended when the paper is flipped around to expose the embossed lines for the student.

Adventure Map: Activity 3

Teaching tip: Provide sufficient time for the student to explore, develop skills, and have fun at each step! Encouraging creativity and personal preferences for drawing as much as possible. Some students may be able to accomplish each step in one session; most students will need several sessions to complete the adventure. This is a very technical part of the adventure, and this may become frustrating depending on the technical aptitude of the students and their screen reader/magnification skills. Please give them enough time and space to solve for technical bugs and typing errors; Google Chrome will do a basic debug of SVG code and tell the student what lines have errors when opening the SVG within the browser.

Debugging and understanding how to change the attribute values are crucial steps to this entire process. Values within the quotation marks should be the only things changing in the code, unless students decide to remove certain styling attributes altogether. There is a lot of potential for typing/syntax errors such as extra or missing quotation marks, the wrong quotation marks being used,

missing or added spaces between the attributes, the equal signs, and the values in the quotation marks, and potentially deleting the closing forward slashes and greater-than signs.

Some examples of common mistakes:

- `stroke-width="4` — missing quotation mark, so this code is broken.
- `stroke-width="4"` — fixed code with ending quotation mark.
- `stroke= "teal"` — space between the equal sign and the value in quotes breaks the code.
- `stroke="teal"` — fixed by removing the space.

1. Introduction

Students will now have a chance to understand how changing values in their code can alter the tactile output of their illustrations from Activity 9, and start to learn how the numerical units map to the actual output on the paper.

2. Think About the Values

- Let students look back through their code and start to hypothesize about what would happen to their drawings as they alter specific values in the attributes of their shapes.
  - Where will shapes end up if they change the x and y coordinates of a rectangle, the cx and cy values of a circle or ellipse, or start plotting out new points for lines and polylines?
  - What happens when they alter the size of the stroke-width or change or remove the fill color?

3. Make the Changes

- Have the students save out a copy of their original SVG files and make their proposed changes to the copy. This way their original file can be preserved to make it easier to revert or assess the changes between the values later on.
- They can get creative, adding or removing shapes, but ultimately they should focus on moving or resizing just a few shapes in their drawings.
- Once they've made the changes and have made sure they have typed everything correctly and that their drawings render in Chrome or Firefox without any errors, emboss their new drawings.

4. Compare and Contrast

- Have students compare and contrast their original embossed drawing with the new one they've just output.
  - Do the positions of the new shapes match with what they thought would happen with their code changes?
  - Did anything surprise them?

  - Does the output make sense between what they had in their original code and what they've altered in the new file?
- Being able to feel the differences between the drawings and mapping how much the units have changed the shapes is critical. Feeling how much a shape moves with subtle or large value differences and how that relates to the SVG coordinate system is what helps unlock the coding barrier between creativity and the final output. Knowing exactly where a shape will end up with specific attribute values builds spatial reasoning and sets them up for future success when building more drawings.

5. Iterate, Iterate, Iterate

- Have the students iterate on their drawings by changing the values again. If students were surprised by the output, have them figure out what they need to change to get the shapes into the exact size and position that they wanted in the first place.
- If students nailed it on their first try, have them iterate again, changing and adding shapes with different values and positions to eventually feel how the numbers and the tactile output relates.
- If the students had a drawing they liked but needed to change something or made a mistake, isn't iterating with code a little easier than redrawing the entire picture?
- Relate how in the real world of design, iterations happen all the time. The first sketch they made may be accepted by a client or a friend, and then once they see the digital version of it, they may want a little change here and there.
- This can be practiced by having each student alter one specific part of their iterated drawing. Give them a very specific piece of direction, such as moving shapes a specific set value in any direction, adding a new shape, removing a shape, etc.
- Continue to emboss out their iterations once they are done with the code, and assess if they have followed the iterative instructions and assess if they are happy with the results. The changes in the code should start to get more comfortable as they read through their SVG files and alter them.

6. Final Goals, TADA!

Students have now made their way through the entire adventure and are now skilled explorers ready to advance further into the wilds of tactile illustration! They've developed great skills and found great tools to keep in their travel packs, and can now create graphics either by hand or via code on the computer.

Students can begin progressing through the rest of BlindSVG.com at their comfort and leisure if they are interested in learning more about the graphics creation process. Have the students think about what they can now create, and challenge them to get out there and try it for themselves!

Extension Activity:

- Have the students go back to Activity 1 of TADA and try creating more tactile drawings or reiterating on the original drawing they made.
  - After going through this entire process and having practiced all these skills, are they finding drawing to be a lot easier?
  - Does it feel a little more natural to take their thoughts and ideas and put them down in a tactile format?
  - Do they feel a little more creative now that they have a new creative outlet?

Thank you so much for exploring with us!

Previous: Activity 2: Digitizing Drawings with SVG Code · Download the whole thing · **Next:** 2D to 3D Overview · **Back to:** Adventure Map

Creating 2D and 3D Tactile Math - Overview

Creating 2D and 3D Tactile Math - Overview

Download this activity

 Supported by a grant from the Northwest Center for Technology Training (CATT-NW) at Washington State School for the Blind.

Introduction

Creating tactile mathematics drawings requires a precision that is challenging to achieve in freehand sketches. In these Activity Plans, we will describe how to use the open-source program OpenSCAD to create 2D and 3D graphics. We also describe how to access our library of over 100 3D printable models which we have created in OpenSCAD to support our science and mathematics books.

OpenSCAD is a computer-aided design (CAD) program that allows a user to create objects with a text-based interface. It is a free, open-source program downloadable from openscad.org. OpenSCAD runs on MacOS, Windows and Linux computers, but not on tablets, phones or Chromebooks. It needs to be installed on the computer (it is not web based). There are various commercial derivatives that have different features, but these are not compatible with the models in our repository and often are adding visual coding features.

Objects are designed in OpenSCAD with text, in a custom language very similar to the C, Java, or Python coding languages. This means that Blind and Low Vision (BLV) users can own the entire workflow, although they will need to wait for the end product to be able to check their design. (Unless they use an AI chatbot or colleague to check the design before committing it to swell paper or a 3D printer.)

OpenSCAD is a good tool to draw precise shapes and show relationships among different shapes, for example in teaching and learning geometry. We do not cover the mathematics behind our models here. We are more focused on the mechanics of drawing rather than why one might want to draw a particular

curve or create a 3D model. For the math background, refer to our books and associated model repositories in the Resource section of TADA! There are two main sections to the TADA! Activities about 3D printing and OpenSCAD:

- Section A: Making SVG Shapes in OpenSCAD
  - Examples of creating 2D mathematical shapes and exporting them to SVG
- Section B: Programming in OpenSCAD
  - A discussion of text coding in general and using OpenSCAD to create 2D and 3D models in particular
- Section C: Guide to Published 3D Printable STEM Models
  - An overview of the 100+ models in our Github repositories, other free community resources, and our books.

Graphics in this Guide

We have a folder of 2D Graphics in SVG format that are created in these activities, for readers who want tactile versions. For that reason we have not included any screenshots and have kept code samples as just indented plaintext for maximum accessibility. We note the .svg filename for each of the line graphics in this guide.

Rather than alt text, which in most cases will be redundant with the name of the simple 2D graphic, we have a short descriptive title (in italics) for each inline figure in this document and note the name of the corresponding svg for a tactile version. We assume that the reader is familiar with making tactile diagrams from svg files.

For the 3D models we discuss, we will have OpenSCAD screenshots or other photographs with alt text.

Other Tools for Tactile Graphs

Other tools exist for creating tactile graphs, notably Desmos. There are articles on their site about Desmos accessibility (including generating Braille equations) and exporting Desmos graphs for embossing. We will not discuss creating accessible graphs here since that territory is well-served by Desmos.

Screen-Readable Equations

If you want to create materials for publication using equations (as opposed to drawings) several tools and workflows exist, although all of them have some drawbacks and challenges in their relationships with authoring tools and then their screen-reader interfaces. We will discuss this in more detail in our Guide to published STEM models and books. Some publishers change equations into images and provide alt text, but this is cumbersome and error prone, and we hope that there are ways to automate the process of creating readable equations.

Lake Pine Braille's Equalize Editor for equations is another free tool to write math equations, but is not a graphics program.

3D Printing

The 3D examples in Activities B.2, B.3 and C are intended to be created by a 3D printer. 3D printing is a complex subject, and we have covered it in depth elsewhere. It is a three-step process:

- Creating a model (the focus of these activities)
- Running a "slicing" program to turn the model into command for the printer
- Actually printing the model.

We developed a guide and lesson plans for 3D geometry objects for another project. Check out the "Teacher's Guide" there, which has an extensive introduction to 3D printing aimed at TVIs as well as links to other resources.

Our book Mastering 3D Printing, 2nd Edition (2020) from Apress (available on Amazon and other resellers) is a more comprehensive guide. The examples in this document were designed to print well on a standard consumer-grade 3D printer without support material or other challenging features. We discuss design decisions for our published models in Section C of this guide.

Tip: 3D Printing for BLV

3D printing can be challenging for anyone, but more so for BLV users since some key parts of the software development chain have limited accessibility. There are several open source slicing programs (called "Slicers") to do the second step, and this requires some knowledge of a 3D printer.

There are some efforts in the community (like 3D Make) to automate some of these away from the user, but it is still early days. Anecdotally we hear that BLV slicer users get by with a mix of AI or colleague review of screenshots during the process of slicing and printing. The group at "3d-Printing-Access" has been sharing practical tips and tricks for non-visual 3D printing, and you might want to check them out. To join, make an account at groups.io and then search on "3d-Printing-Access" to find the group.

Previous: Activity 3: Iterating on SVG Drawings & Beyond · Download the whole thing · **Next:** Section A: Making 2D Shapes · **Back to:** Adventure Map

Section A: Making 2D Shapes in OpenSCAD

Section A: Making 2D Shapes in OpenSCAD

Download this activity

These activities show you how to use the OpenSCAD program to create 2D shapes and directly export them as an SVG. OpenSCAD is an open-source, free computer-aided design (CAD) program. It can create 2D shapes in SVG or DXF format (laser/vinyl cutters and embossers), and 3D shapes for 3D printing. This activity focuses on very basic activities. The activities in the "Programming in OpenSCAD" section discuss OpenSCAD and its features in more depth. We also provide links to more resources about coding in general there.

Objectives

- Learn to create precise geometrical shapes in OpenSCAD
- Learn how to combine shapes in various ways and to translate, rotate and mirror them
- Learn how to export shapes in SVG format

Materials

Download and install OpenSCAD on a computer running MacOS, Windows or Linux (not a tablet, iPad, Chromebook or other mobile device). The program (and good documentation) is available at openscad.org. All these activities assume that OpenSCAD has been installed on your computer.

Other Resources

If you need more help, our book Make: Geometry has more detailed OpenSCAD startup instructions. Along with our other math books and a teacher's guide, it is available in the Maker Shed and other book retailers.

One of the creators of OpenSCAD also collaborated on the book Programming with OpenSCAD: A Beginner's Guide to Coding 3D-Printable Objects, by Justin Gohde and Marius Kintel, available from the publisher at nostarch.com or on Amazon.

If you like to watch videos instead, the OpenSCAD team created tutorial videos linked on their site. They do not have formal video descriptions per se but they talk through what they are doing on the screen as they do it.

- There is an OpenSCAD introduction video about the same level as the beginning of our 3D materials. It is roughly nine and a half minutes long.
- For more depth, there is a YouTube playlist of 27 videos that start with downloading and go through progressively more advanced materials.

Ideas for Carrying Out this Activity

OpenSCAD is accessible with a screen reader. One option is to create a file with the examples we have created here and allow them to compare those with tactile outputs. Then, encourage students to change and combine examples, predict what will happen, and then export their version to SVG for a tactile print.

Alternatives

Teachers who can use a visual interface might find it easier to create models in other drag and drop software (such as Tinkercad, available at tinkercad.com). OpenSCAD's great strength is that it makes it possible to create accurate 2D and 3D models with mathematical relationships. For example, it is pretty easy to draw a rectangle and rotate it 45 degrees.

Previous: 2D to 3D Overview · Download the whole thing · **Next:** Activity A.1: Create a Square · **Back to:** Adventure Map

Activity A.1. Create a Square in OpenSCAD

Activity A.1. Create a Square in OpenSCAD

Download this activity

This activity creates a square 25 mm (about an inch) on a side. All the activities here assume that OpenSCAD is open and working, and that you have launched it with the usual graphical user interface.

Steps:

- Go to the File menu, and select New File.
- Navigate to the Editor window.
- In the Editor window, type:

```
square(25);
```

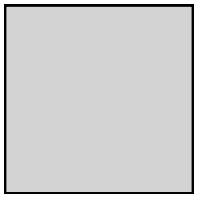

Figure 1: A square, file square25.svg

A square, file square25.svg

You will note that there is a semicolon at the end of the command. OpenSCAD uses a programming language that is sort of like the C, Python or Java programming languages, with some quirks. You can do a lot by bearing just a few rules in mind which we will describe in later activities. For now, just think of it that any 2D object description needs to end in a semicolon.

Also, OpenSCAD uses three different types of parentheses or brackets that mean different things. Make sure you follow our examples exactly. This simple example uses just parentheses; others will use square or curly brackets.

Tip: Troubleshooting

If the square doesn't show up, this is a good time to reinforce that coding is precise and predictable. Oftentimes things don't show up as planned due to a syntax error. Have your student read back their code, character by character, this verbal check will help you find syntax issues.

Continuing on with the steps to create our SVG showing a square:

- In the main Menu bar along the top, select Save and give the model an appropriate name ending in .scad, like square.scad.

Rendering and Exporting an SVG

The file ending in .scad can be edited in OpenSCAD going forward. (SVG files can be opened in OpenSCAD, but not edited.) Now that we have saved the file we can try rendering, exporting and printing it.

- In the Design menu, click Render. This creates an exportable version of the file.
- In the File menu, select Export and then Export as SVG.
- The file should now be exported to a location you specify and ready for tactile printing. It should produce a square 25 mm on a side (just under an inch).

Activity 2 and 3 have the same workflow. You will just be typing different things into the editor.

Teaching Tip: Editing SVGs

Existing SVG files can be imported into OpenSCAD, as described on the OpenSCAD manual "SVG Import" page. However, they can not be edited per se. Technically they can be combined with other shapes, but this is very case-dependent and not recommended. Shapes generated by OpenSCAD are shown with a solid outline and a light grey fill, as we show in this document. There are no options to change it in OpenSCAD itself.

> **Note:** the image above (`square25.svg`) is embedded from the original site and currently has empty alt text — worth writing real alt text describing the shape when you migrate the actual image file over.

Previous: Section A: Making 2D Shapes · Download the whole thing · **Next:** Activity A.2: Basic Shapes · **Back to:** Adventure Map

Activity A.2. Basic Shapes in OpenSCAD

Activity A.2. Basic Shapes in OpenSCAD

Download this activity

Now that we have tried out the workflow for creating a square, we can try out other built-in 2D shapes as well as adding, subtracting or intersecting two or more shapes. We used a square the most straightforward way, by just specifying a side. OpenSCAD also lets you create "squares" (rectangles, really) with sides that are different lengths. You can also create circles, and general polygons.

Steps:

All the activities here assume that OpenSCAD is open and working, and that you have launched it with the usual graphical user interface.

- Go to the File menu, and select New File.
- Navigate to the Editor window.
- In the Editor window type the code to create an object (code given in examples that follow).
- In the main Menu bar along the top, select Save and give the model an appropriate name ending in .scad, like rectangle.scad.

Rendering and Exporting an SVG

The file ending in .scad can be edited in OpenSCAD going forward. (SVG files can be opened in OpenSCAD, but not edited.) Now that we have saved the file we can try rendering, exporting and printing it.

- In the Design menu, click Render. This creates an exportable version of the file.
- In the File menu, select Export and then Export as SVG.
- The file should now be exported to a location you specify and ready for tactile printing. It should produce the shapes as described in what follows.

Code to Create General 2D Shapes

The basic 2D shapes in OpenSCAD are squares, circles and polygons. With various tricks we can generate many types of shapes from these. Let's look at

generating a few common shapes from these options. We already saw how to draw a square in the previous activity. Suppose we want a rectangle instead? We would define a "square" with different length and width by typing something like this in the Editor window:

```
square([25, 100]);
```

creates a rectangle 25 mm long and 100 mm high.

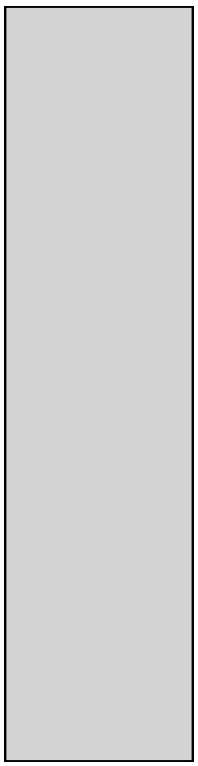

Figure 1: A rectangle 25 by 100, file square25,100.svg

A rectangle 25 by 100, file square25,100.svg

A circle is defined several different ways. One is to define the radius, like this:

```
circle(r = 25);
```

which would give us a circle of radius 25 mm. In fact we can just say:

```
circle(25);
```

which accomplishes the same thing. (There are other ways to specify a circle, but we will leave you with just these two.)

A circle of 25 mm radius, file circle25.svg

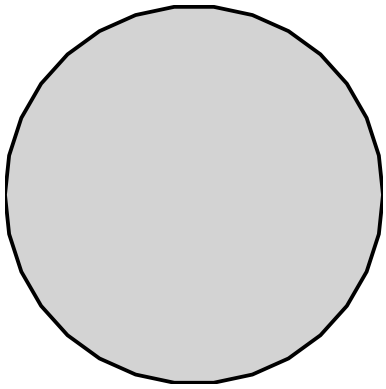

Figure 2: A circle of 25 mm radius, file circle25.svg

Suppose instead we wanted a regular polygon - that is, a shape with N sides of equal length. A "circle" in OpenSCAD is really a lot of tiny straight lines, called facets. With enough facets, every line is short enough that they seem to be making a continuous curve. But if instead we said, "draw a circle but with only three facets" we would get a triangle, like this:

```
circle(r = 25, $fn = 3);
```

where $fn = 3 tells us that the number of facets is 3, and the radius is the distance from the center of the shape to a vertex. (This is called the "inscribed polygon" of the circle of the same radius; it is the biggest polygon that can be drawn that will fit entirely inside the circle.) If $fn = 5, we would get a pentagon, and so on.

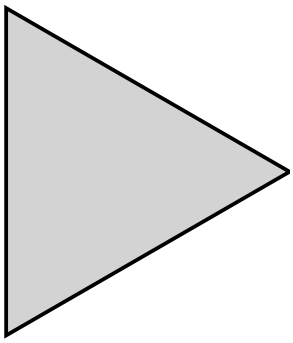

Figure 3: A "circle" turned into a triangle of radius 25, file radius25.svg

A "circle" turned into a triangle of radius 25, file radius25.svg

Teaching Tip: Experiment

Encourage students to experiment with slight changes in numbers or parameters (try a different number for the radius or a different number of facets). Use these small adjustments to build confidence and explore cause/effect relationships in the code.

Other, more general polygons can be drawn by creating a list of points that define its vertices. For example, to draw a triangle whose vertices are the points (0, 0), (100, 0), and (50, 100) we would write:

```
polygon(points = [ [0, 0], [100, 0], [50, 100] ]);
```

Note that each (x, y) coordinate pair is enclosed in a pair of square brackets, and the whole set is inside another set of square brackets and a pair of parentheses. Be careful not to mix those up since they mean different things in OpenSCAD.

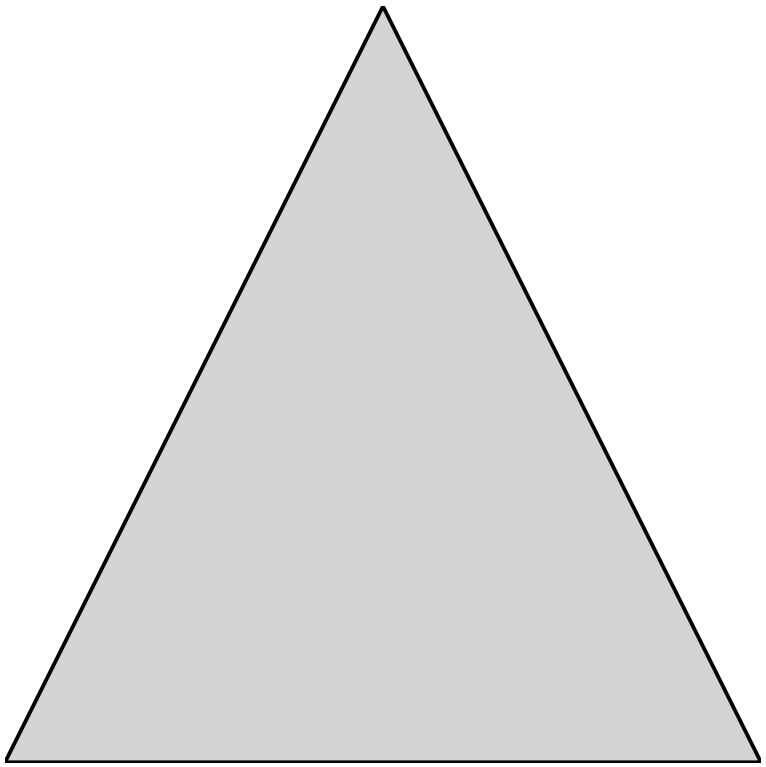

Figure 4: A triangle created with the polygon function, file polygon.svg

A triangle created with the polygon function, file polygon.svg

This polygon can be as complex as we want. By default the first and last points in the list will be connected. There are many more options and tricks in the OpenSCAD manual "2D Subsystem" page.

Note: the images above are embedded from the original site with empty alt text — worth writing real descriptive alt text when you migrate the actual image files over.

Previous: Activity A.1: Create a Square · Download the whole thing · **Next:** Activity A.3: Rotating, Translating, Scaling · **Back to:** Adventure Map

Activity A.3. Rotating, Translating, Scaling and Mirroring Shapes

Activity A.3. Rotating, Translating, Scaling and Mirroring Shapes

Download this activity

The real power of OpenSCAD is not in creating individual shapes one at a time. Instead you can create complex drawings with as much structure as you like. Each of the individual shapes we have talked about in Activities 1 and 2 can be altered by using additional OpenSCAD functionality to modify them. In the Programming in OpenSCAD section we get into the ways to combine shapes.

Steps:

All the activities here assume that OpenSCAD is open and working, and that you have launched it with the usual graphical user interface.

- Go to the File menu, and select New File.
- Navigate to the Editor window.
- In the Editor window type the code to create an object (code given in examples that follow).
- In the main Menu bar along the top, select Save and give the model an appropriate name ending in .scad, like rectangle.scad.

Rendering and Exporting an SVG

The file ending in .scad can be edited in OpenSCAD going forward. (SVG files can be opened in OpenSCAD, but not edited.) Now that we have saved the file we can try rendering, exporting and printing it.

- In the Design menu, click Render. This creates an exportable version of the file.
- In the File menu, select Export and then Export as SVG.
- The file should now be exported to a location you specify and ready for tactile printing. It should produce the shapes as described in what follows.

OpenSCAD Conventions

OpenSCAD uses a Cartesian coordinate system (x and y axes in 2D, and x, y, and z in 3D). This means we have two right-angle axes (or three, in 3D) crossing at the point x = 0 and y = 0. This is in the lower left-hand corner of the pair of axes that follows, where they cross. (Negative numbers would be to the left for negative x values, and down for negative y values.)

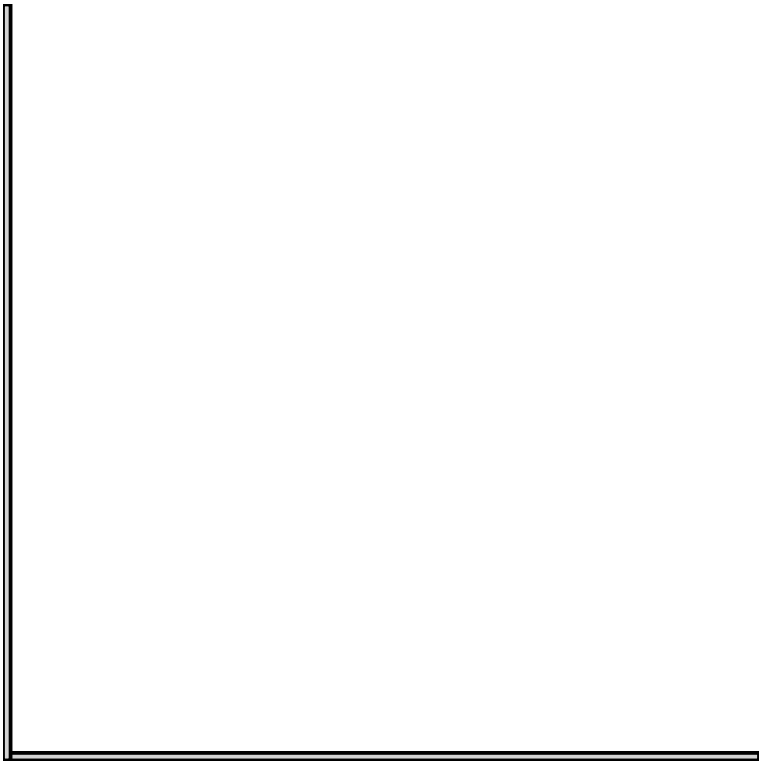

Figure 1: A pair of axes, only showing positive directions, file axes.svg

A pair of axes, only showing positive directions, file axes.svg

By default, polygons are drawn with one vertex at 0, 0, but adding the parameter center = true like this:

```
square([5, 10], center = true);
```

will cause them to be drawn around the (0, 0) point instead. Circles are always centered at (0, 0).

Drawing Graph Axes

OpenSCAD does not export any axis markings. You can always add a few lines to your code to draw them. For example, these three lines will draw a pair of axes 100 mm long and 1 mm wide showing positive x and y:

```
axislength = 100;

square ([axislength, 1]);
square ([1, axislength]);
```

You can always add this code to your other objects so you can see what changed. The pair of axes in file axes.svg shows axes drawn this way.

Rotation

Suppose we want to rotate our square 30 degrees. Since we are rotating a 2D object, we want to keep it in the plane of the paper. It turns out that this means that we have to rotate it only around the third (z) axis. Rotating around the other axes would take us out of the plane of our paper.

The functions that modify an object are placed in front of them, without a semicolon. To rotate around the z axis 30 degrees, we would write:

```
rotate([0, 0, 30]) square([15, 20]);
```

The first two zeroes in the rotate function show that we are not rotating around the x and y axis However this rotates using the line that forms the bottom of the rectangle:

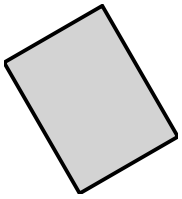

Figure 2: A square rotated by sweeping the line that forms its bottom (y = 0) line by 30 degrees, file rotatesquare.svg

A square rotated by sweeping the line that forms its bottom (y = 0) line by 30 degrees, file rotatesquare.svg

If we wanted to instead rotate around the center of the rectangle, we would write:

```
rotate([0, 0, 30]) square([15, 20], center = true);
```

This would create a square rotated parallel to the one in rotatesquare.svg, but shifted by 7.5 mm in the x direction. (This is in file *rotatesquarecenter.svg* - since we are centering graphics in this text, it would look the same here.)

Translation

Similarly, we can move an object left, right, up or down with the translate([x, y]) function. This would take our rectangle and shift it 5 millimeters to the right (you can see this in *file translatesquare.svg*):

```
translate([5, 0, 0]) square([15, 20]);
```

We can also use multiple modifiers to translate and then rotate, or vice versa. Translating and then rotating gives a different result, since the axis of rotation moves with the object. Try each of these two lines to see the difference.

```
rotate([0, 0, 120]) translate([15, 0, 0]) square([15, 20]);

translate([15, 0, 0]) rotate([0, 0, 120]) square([15, 20]);
```

Note that the order matters, and OpenSCAD executes the functions from right to left - creating the rectangle in both cases, but in the first one translating then rotating, and the reverse for the second. Both squares are shown relative to each other here:

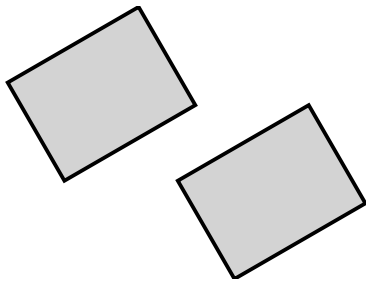

Figure 3: The two examples of rotating and translating a square shown together, file rotatetranslatesquare.svg

The two examples of rotating and translating a square shown together, file rotatetranslatesquare.svg

Mirroring

If we want to reflect an object across a line running along an axis, we can use the mirror function. We can reflect 2D objects in x and y. (The mirror function, though, expects three arguments even in 2D.) We use a zero for each axis we do not want to mirror, and a 1 in axes we do want to mirror. To mirror our square about the x axis, we would use:

```
mirror([1, 0, 0]) square([15, 20]);
```

Note that

```
mirror([1, 0, 0]) square([15, 20], center = true);
```

is the same as

```
square([15, 20], center = true);
```

Since OpenSCAD then mirrors about the center of this (symmetrical) object. All of these functions can interact in subtle ways. Playing with them is a good way to learn to think about rotation, mirroring and translation as mathematical concepts, too. You can see them respectively in *mirrorsquare.svg* and *mirrorsquarecenter.svg*.

Scaling

The scaling function is handy to create new shapes. For example, if you want an ellipse, you can do this:

```
scale([1, 2, 0]) circle(15);
```

which gives an ellipse that has a 15 mm semiminor axis in x and a 30 mm semimajor axis in y. (There is no built-in ellipse datatype.)

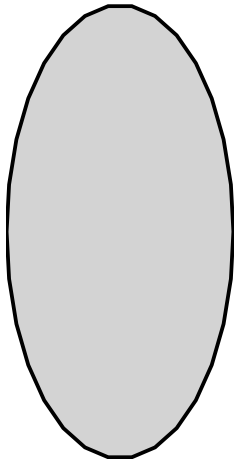

Figure 4: An ellipse from scaling a circle in the y-dimension, file scalecircle.svg

An ellipse from scaling a circle in the y-dimension, file scalecircle.svg

As with combinations of the others, OpenSCAD evaluates from the inside out (later statements are considered to be "enclosed" in the ones that precede them in a line), and the following two lines give a different result. The first one will rotate and then scale our square, resulting in a rhombus. The second will stretch the square into a rectangle, then rotate the rectangle.

```
scale([1, 2, 0]) rotate([0, 0, 45]) square(20, center = true);

rotate([0, 0, 45]) scale([1, 2, 0]) square(20, center = true);
```

You can see these two options relative to each other here:

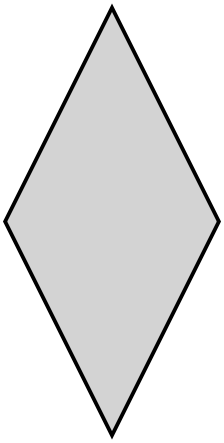

Figure 5: A square rotated and then scaled to create a rhombus, file scalerotatesquare.svg

A square rotated and then scaled to create a rhombus, file scalerotatesquare.svg

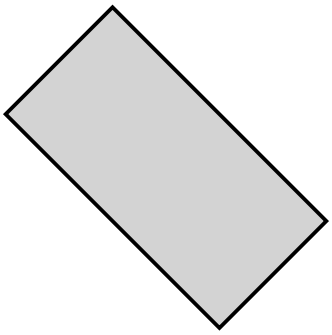

Figure 6: A square scaled then rotated to create a rectangle oriented diagonally, file rotatescalesquare.svg

A square scaled then rotated to create a rectangle oriented diagonally, file rotatescalesquare.svg

In general, it is prudent to create small test cases to try out combinations of operations first and then incorporate them incrementally into more complex designs.

Teaching Tip: Practice

Have students act out a "rotation", "translation" or "mirroring" with physical objects to help them understand what those commands actually do.

Note: the images above are embedded from the original site with empty alt text — worth writing real descriptive alt text when you migrate the actual image files over.

Previous: Activity A.2: Basic Shapes · Download the whole thing · **Next:** Activity B.1: Adding & Subtracting · **Back to:** Adventure Map

Activity B.1. Adding and Subtracting Objects

Activity B.1. Adding and Subtracting Objects

Download this activity

OpenSCAD is a constructive-geometry CAD program. What that means is that a user defines an object and then modifies it in various ways, like scaling it, rotating, and so on. In this activity we are going to look at ways to merge objects together, or to use one object as a shaped hole in another. We can also save just the intersection of two objects.

As we saw in the "Making 2D Shapes in OpenSCAD" section of this guide, the fundamental structure of a model is an object, possibly modified. Let's look at this rotated rectangle again, and deconstruct it a little.

```
rotate([0, 0, 30]) square([15, 20]);
```

Note that we have a set of square brackets nested inside a set of parentheses. The square brackets tell us that we have a set of numbers, like the angles of rotation in three axes. The parentheses enclose however many sets of numbers we have for that particular object or modifier. If we have

```
square([15, 20], center = true);
```

we can see that the dimensions of the square and the optional item that it is supposed to be centered at the origin are both inside the parentheses.

We learned in the 2D set of activities that objects are modified starting from the closest modifier to the farthest, or right to left. In this case:

```
scale([1, 2, 0]) rotate([0, 0, 45]) square([15, 20], center = true);
```

our square was rotated and then scaled. Sometimes it does not make a difference, but usually it does.

Adding Shapes

Suppose we wanted to create an "L" shape with two rectangles. We could do that two different ways: by either adding two rectangles together, or by subtracting a smaller rectangle from a larger one. Adding two objects is accomplished by using the union() operator, which looks like this:

```
union() {
  square([15, 45]);
  square([45, 15]);
}
```

We enclose the two things we want to add together inside curly braces. By convention people indent what is inside the braces, plus the second brace, to make it easier to read.

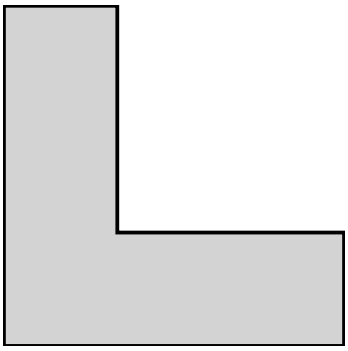

Figure 1: L-shaped figure, file union.svg

L-shaped figure, file union.svg

Subtracting Shapes

We can create the identical figure by subtracting using the difference() operator on a 15 by 15 square from the upper right corner of a 45 by 45 square, like this:

```
difference() {
  square([45, 45]);
  translate([15, 15]) square([30, 30]);
}
```

Notice that we had to translate the square before differencing. The second thing inside the difference curly braces is the thing you subtract, just like the thing after the minus sign is the thing you subtract. We created a file *difference.svg*, and if you create it you will find that it is the same figure as *union.svg*.

Intersecting Shapes

Finally, we can intersect two objects. If we intersected our 45 by 45 square with our 15 by 15 one, we would just get the smaller square since they overlap completely. If we translate the square a bit more, that will no longer be the case. Try it and check out how the results vary. Of course, if you translate the smaller square far enough, the intersection will just be nothing at all. (File *intersection.svg* shows this square.)

```
intersection() {
  square([45, 45]);
  translate([15, 15]) square([30, 30]);
}
```

Other OpenSCAD Conventions

Extra spaces and carriage returns are optional in OpenSCAD. You can put more code on one line, but it can make your model hard to read and understand if you are looking at it visually. The previous set of shapes would look like this all on one line:

```
difference() { square([45, 45]); translate([15, 15]) square([30, 30]); }
```

One place you cannot add spaces is inside one of the names of shapes ("sq uare" is not legal). OpenSCAD is also case-sensitive, so you need to be careful not to type Difference or DIFFERENCE, which will not work.

If you are creating a model that is getting a little complicated, you can also leave notes to yourself, called "comments." If we start a line with two forward slashes, like this:

```
// This is a comment
```

OpenSCAD ignores everything up to the next carriage return. If you have a longer block of text covering multiple lines you can start your long block with /* and end it with */ and OpenSCAD will ignore the whole thing. This can be handy if you want to check just part of your model.

```
/* This is also a comment

Just a longer one

*/
```

More Complex Shapes

There is no limit to how many things you can add, subtract or intersect. To do that, we can nest one or more activities inside another. You can also translate, rotate or mirror a union, difference, or intersection. For example, the following code creates a circle with an L-shaped bit taken out of its upper right hand side.

```
difference() {
  circle(50);
  translate ([15, 15]) union() {
    square([15, 45]);
    square([45, 15]);
  }
}
```

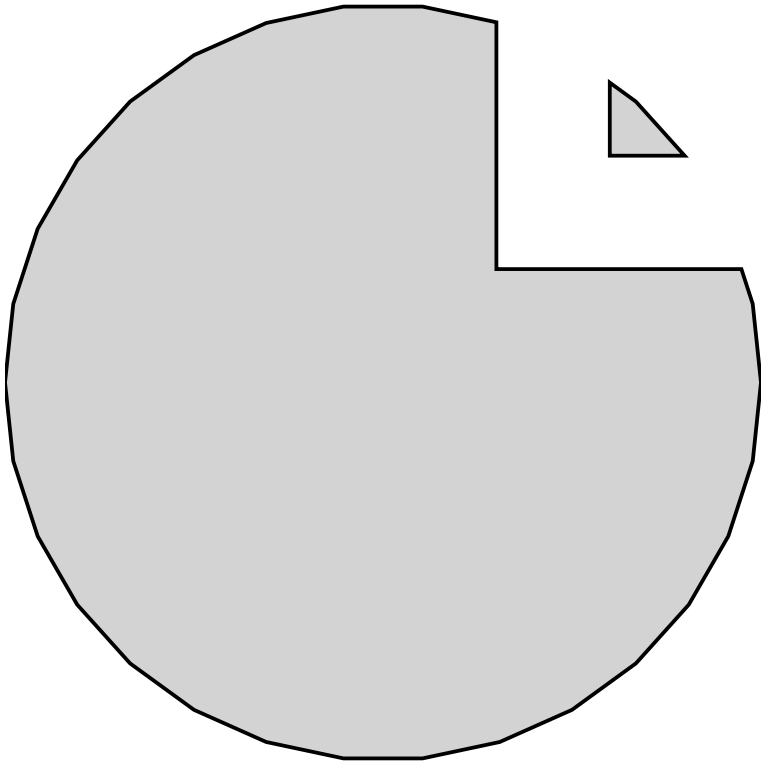

Figure 2: A circle with an L-shaped bite out of the upper right, file complexdifference.svg

A circle with an L-shaped bite out of the upper right, file complexdifference.svg

Teaching Tip: Practice

Use hands-on manipulatives, like foam shapes or Legos, to physically demonstrate union, difference and intersection before jumping into code. This tactile comparison helps students connect the code to spatial thinking and reinforces vocabulary like "subtract" or "intersect".

OpenSCAD has other capabilities like being able to create repeated sets of objects. To do that, we need a programming construct called a loop. Let's create a set of five circles marching across our page. We enclose the thing we want to do multiple times in curly brackets, just as we did for other manipulations.

Next we set up our loop, which starts with the word "for" and shows us how many times we want to do something, a little indirectly. Our loop uses a "loop counter" (the letter "i" in the example that follows) The loop runs a total of five times (starting at zero and ending at four), and it looks like this:

```
for(i = [0:1:4]) {
  translate([i * 10, 0]) circle(5);
}
```

The first time i = 0, and we use that value to see how much to translate (i times zero, or zero). The next time we translate one times 10, (10 mm) and so on. When we have gone through the fifth time, i = 4, and we translate 40 mm to draw our last 5 mm circle. You can leave out the center number (how much to step) if you are stepping by 1 each time, but we included it here for completeness.

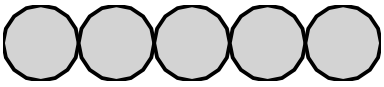

Figure 3: Five circles in a line, touching each other, file forloop.svg

Five circles in a line, touching each other, file forloop.svg

Note that an asterisk (*) means "multiply" in OpenSCAD, as it does in many other programming languages.

There are many other functions and capabilities that you can learn more about in the OpenSCAD User Manual. We will introduce a few more as we now move into 3D shapes.

Variables and Conditionals

Like most programming languages, OpenSCAD lets you assign values to named variables. This lets you assign value once, then use it in multiple places. If you later decide you want to change that value, you only need to change it in one

place. Setting a variable is done by typing the name of the variable, an equal sign, then the value you want to assign to it. This assignment is directional, so unlike in math, a = b does not mean the same thing as b = a. So for instance:

```
radius_inner = 10;
```

Would let us make a bunch of objects that all need a radius variable. One quirk of OpenSCAD relative to other programming languages is that variables can only be set once in OpenSCAD. See the OpenSCAD manual page on general syntax for more.

A variable may be assigned a value that is a number (for programmers a "floating point" number), a string (a series of characters, usually used for a word or phrase), a range (see the for loop example above), or a vector (a series of values stored in one variable, also known as an array, most commonly used for [x, y] or [x, y, z] coordinates).

OpenSCAD also has conditional statements (if … else) statements, but the rules about variables being assigned once mean that you cannot do some things that would be common in other programming languages. An if statement starts with the word "if", followed by a test condition in parentheses. What follows those parentheses will only be executed if the condition evaluates as true. This is optionally followed by an "else" statement, which is executed if the condition was false. For example:

```
a = 3;
if (a > 2) circle(5);
else square(10, center = true);
```

This will create a circle, since a = 3 which is greater than 2. If a was equal to 1, we would get a square. In other programming languages you might see something like the following, which will result in an error in OpenSCAD:

```
a = 3;
if (a > 2) b = 5;
else b = 10;
```

The same thing can be accomplished using a construction called the "ternary operator". This operator exists in most programming languages, but is rarely used in most of them.

```
a = 3;
b = (a > 2) ? 5 : 10;
```

This can be read as: b equals 5 if a is greater than 2, else b will equal 10. More information about how to use the ternary operator in OpenSCAD can be found in the OpenSCAD Documentation.

Math Functions

OpenSCAD comes with built-in math functions, including taking a square root (sqrt(4) equals 2, for example), raising a number to a power (pow(2, 3) will return the third power of 2, or 8). For example to create a circle whose radius is the square root of 2, we would write

```
circle(sqrt(2));
```

You can find the full list, and the relevant parameters, on the OpenSCAD "Mathematical Functions" manual page.

> **Note:** the images above are embedded from the original site with empty alt text — worth writing real descriptive alt text when you migrate the actual image files over.

Previous: Activity A.3: Rotating, Translating, Scaling · Download the whole thing · **Next:** Activity B.2: Moving from 2D to 3D · **Back to:** Adventure Map

Activity B.2. Moving from 2D to 3D

Activity B.2. Moving from 2D to 3D

Download this activity

Everything we have seen so far has been aimed at two dimensions, creating an output in SVG format for a 2D tactile drawing. Now, we will move on to 3D objects for a 3D printer. OpenSCAD has 3D shapes that correspond to the 2D one (for example, sphere versus circle) and we will get to those in the next Activity. The easiest transformation is to just extrude the shape into the third dimension.

OpenSCAD has two types of extrusion: linear_extrude() and rotate_extrude(). The linear_extrude function takes a 2D shape and, well, extrudes it into the third dimension by a specified thickness. Imagine you had a cookie cutter the shape of your original 2D shape: the extruded version would be the cookie.

Steps:

All the activities here assume that OpenSCAD is open and working, and that you have launched it with the usual graphical user interface.

- Go to the File menu, and select New File.
- Navigate to the Editor window.
- In the Editor window type the code to create an object (code given in examples that follow)
- In the main Menu bar along the top, select Save and give the model an appropriate name ending in .scad, like extrusion.scad.

Rendering and Exporting a 3D Printable File

The file ending in .scad can be edited in OpenSCAD going forward. (SVG files can be opened in OpenSCAD, but not edited.) Now that we have saved the file we can try rendering, exporting and printing it.

- To get a 3D print rather than a 2D tactile drawing, we need to export a different file. Go to the File menu and then Export and select Export to STL. STL files are what a 3D printer requires.

- For more on 3D printing, see the "3D Printing" section in the Introduction of this guide.

Teaching Tip: File Type

Always connect the change in file type to the change in purpose (.svg for tactile drawings and .stl for 3D printing). Have students talk through when and why you'd use each format, don't assume the distinction is obvious.

Linear Extrusion

If we wanted to create a 3D printed L shape like we had in the previous Activity, we would use OpenSCAD code like this:

```
linear_extrude(height = 5, scale = 1.0) difference() {
  square([45, 45]);
  translate([15, 15]) square([30, 30]);
}
```

Note that we just add the linear_extrude function to the front of our model (enclosing what comes after it), as if it was a scale or translate function. The height parameter (which has to be called that, case sensitive) tells us that the resulting L will be 5 mm thick, and the scale parameter says that it will not get bigger or smaller as we extrude it. For example, if we make scale = 0.5,

```
linear_extrude(height = 5, scale = 0.5) difference() {
  square([45, 45]);
  translate([15, 15]) square([30, 30]);
}
```

Now the L gets smaller as we go up in the z dimension. (You can omit the scale parameter if it equals 1, which is the default, but we used it here to make the point.) The linear_extrude function has fancier parameters as well which you can explore, but this capability should be enough for now. If you are extruding in other than the z direction you may need to add other parameters.

Rotating Extrusion

The rotate_extrude module takes a 2D shape in the x/y plane and first rotates it to the x/z plane, then it sweeps that shape around the z axis to create a 3D shape. This is useful for creating toroids (like a doughnut) and similar shapes. By default, this will sweep the shape around 360 degrees to meet up with itself, but you can optionally use the "angle" parameter to sweep the shape only part way around the axis. Note that the rotate_extrude module requires all of the 2D geometry to be in either the positive x or negative x coordinates. It will fail on geometry that crosses x = 0.

```
rotate_extrude() translate([25, 0, 0]) circle(20);
```

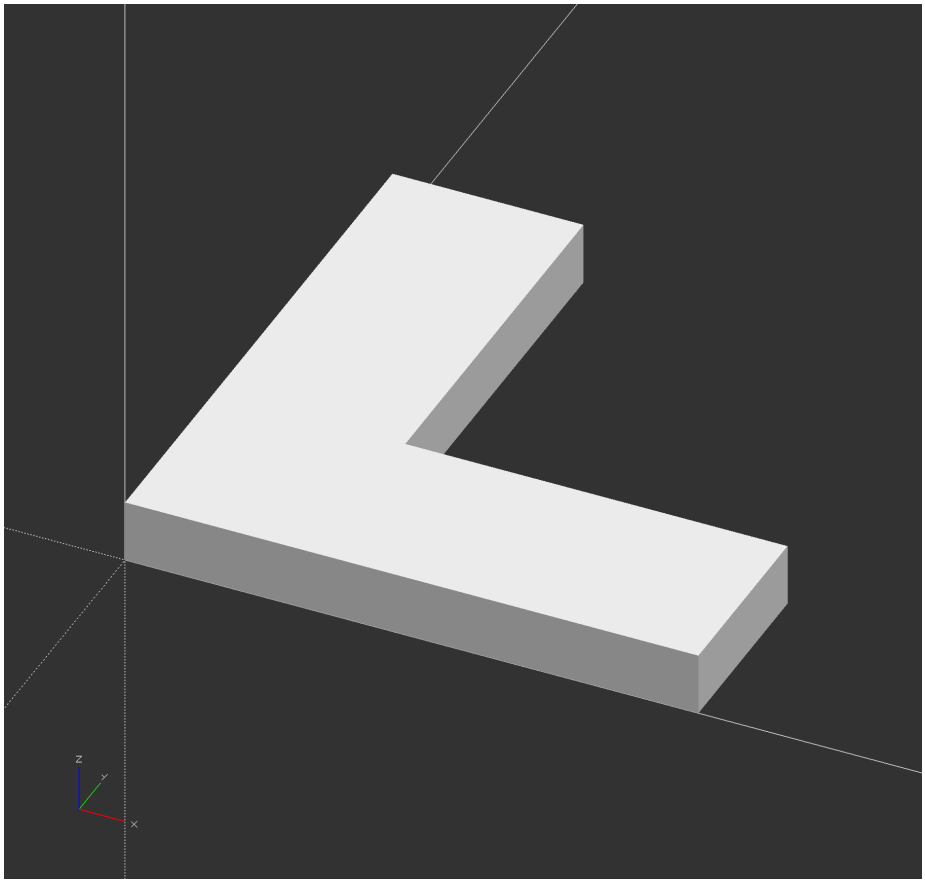

Figure 1: An OpenSCAD screenshot which shows a 3D shape that looks like a three-dimensional letter “L”, parallel to the xy plane.

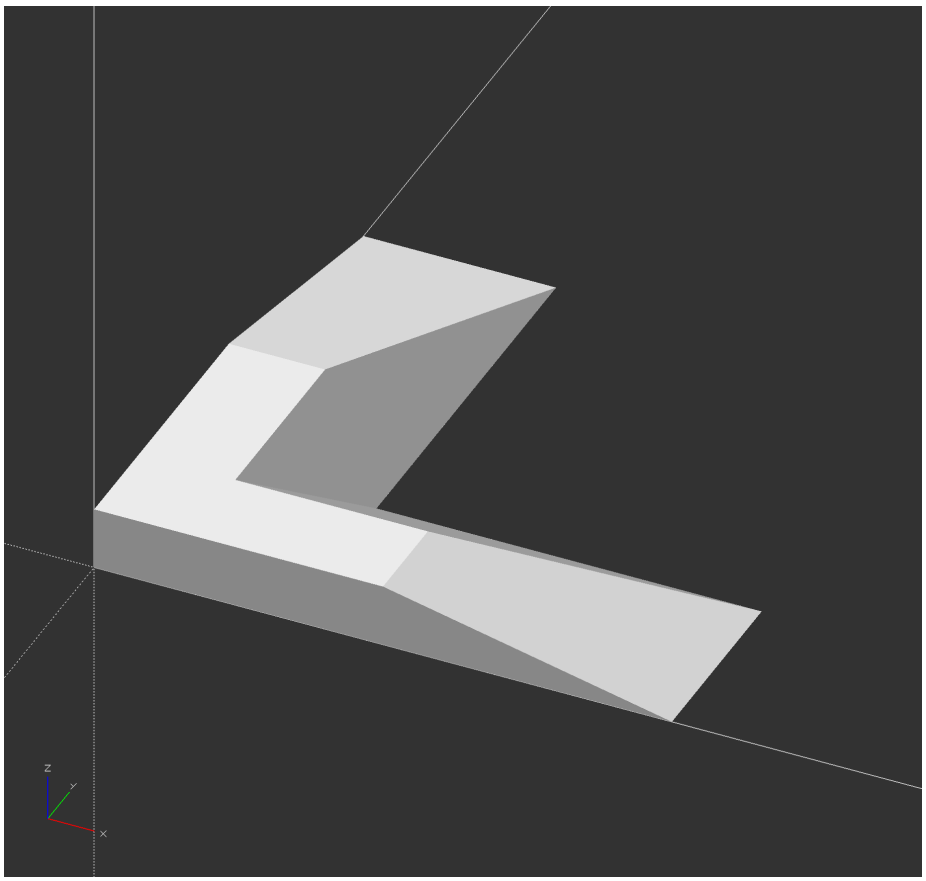

Figure 2: An OpenSCAD screenshot which shows a 3D shape that looks like a three-dimensional letter “L”, as in the previous screenshot, but now the L gets smaller in the z dimension until it is half the size at the highest z. The lower left corner of the L shape is the same for both shapes.

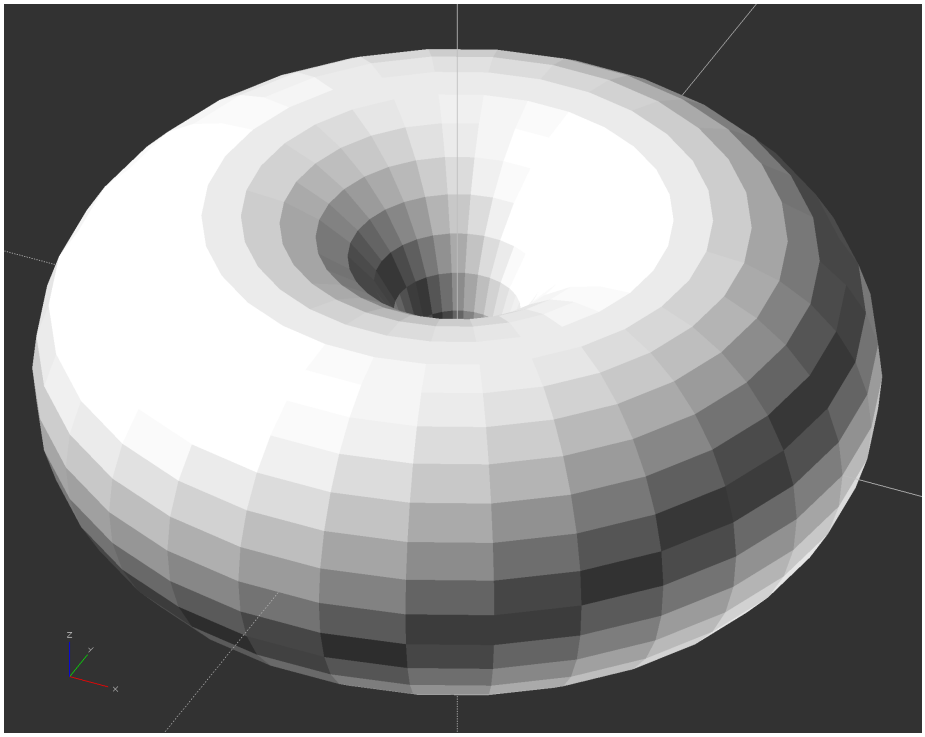

Figure 3: A torus, or doughnut shape, in OpenSCAD

If we now introduce the angle parameter, we can just get part of our doughnut.

```
rotate_extrude(angle = 270) translate([25, 0, 0]) circle(20);
```

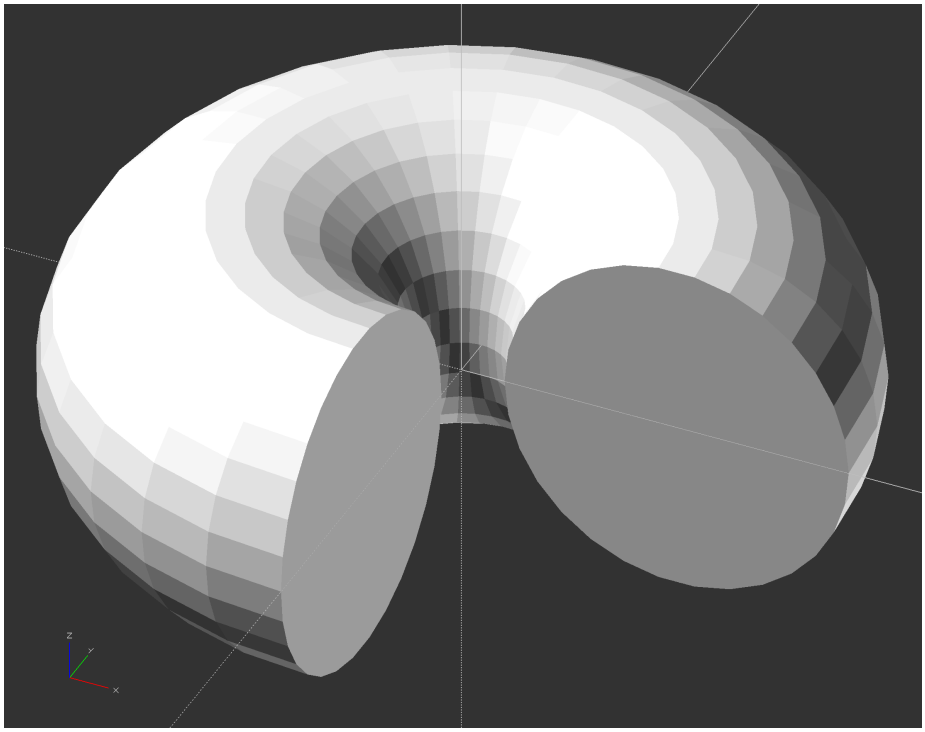

Figure 4: A partial torus, as if someone took a bit out of a doughnut.

> **Note:** the images above already have real descriptive alt text on the original site (unlike the SVG diagrams in earlier activities) — carry that alt text over as-is when you migrate the actual image files.

Previous: Activity B.1: Adding & Subtracting · Download the whole thing · **Next:** Activity B.3: True 3D Models · **Back to:** Adventure Map

Activity B.3. True 3D models

Activity B.3. True 3D models

Download this activity

In addition to the 2D basic shapes we saw back in Activity A, OpenSCAD has 3D primitives like cube and cylinder that work similarly, but are of course three-dimensional. The basic 3D shapes are the cube, cylinder, sphere, and polyhedron. Each one is pretty similar to the related two-dimensional shape.

Steps:

All the activities here assume that OpenSCAD is open and working, and that you have launched it with the usual graphical user interface.

- Go to the File menu, and select New File.
- Navigate to the Editor window.
- In the Editor window type the code to create an object (code given in examples that follow)
- In the main Menu bar along the top, select Save and give the model an appropriate name ending in .scad, like extrusion.scad.

Rendering and Exporting a 3D Printable File

The file ending in .scad can be edited in OpenSCAD going forward. (SVG files can be opened in OpenSCAD, but not edited.) Now that we have saved the file we can try rendering, exporting and printing it.

- To get a 3D print rather than a 2D tactile drawing, we need to export a different file. Go to the File menu and then Export and select Export to STL. STL files are what a 3D printer requires.
- For more on 3D printing, see the "3D Printing" section in the Introduction of this guide.
- In "Advanced Features" at the end of this Activity, we discuss techniques for creating both 2D and 3D shapes, which you can find in detail in Activity A.1 and B.2, respectively.

Basic 3D Shapes

To make a rectangular solid, we use the cube() function. Similar to the square() function, it is called a "cube" but can have unequal sides, and be centered around the origin or not. For example

```
cube([5, 7, 10]);
```

will be a rectangular solid 5 mm long in the x-dimension, 7 mm in y, and 10 high in the z dimension. 3D shapes can be combined the same way as 2D ones (with union(), difference(), and intersection()) and can be scaled, rotated and so on (but all three axes need to be specified). Creating a cylinder is similar:

```
cylinder(h = 5, r = 10);
```

Will create a cylinder of radius 10 mm and height 5 mm. However, there is also a special case of a cylinder that creates a cone:

```
cylinder(h= 5, r1 = 10, r2 = 0);
```

This creates a short, fat cone 5 mm high, 10 mm radius at the base, and a point on top. The two radii represent the base and top of the cone. The top is pointy, and thus has a radius of zero.

OpenSCAD's polyhedron module is a bit more complicated with more to think about, with potential pitfalls described on the manual page.

By rotating and translating these shapes in three dimensions, you can quickly build up 3D objects.

Tip: Simple to Complex

Remind students that 3D coding is still all about shapes and structure. Just like in 2D, we build up from primitives, only now there's an extra axis to think about. Start with code that creates one simple shape, then add complexity layer by layer.

3D House Example

The example that follows is a simple model of a house with a sloped roof, a cylindrical chimney, and an open doorway in the front. It is created with the following code:

```
difference() {
  union() {
    translate([-10, -10, 0]) cube(20);
    translate([0, 0, 20]) rotate([90, 0, 0]) cylinder(r = 10, h = 20, center = true, $fn = 4
    translate([7, 0, 0]) cylinder(r = 2, h = 28);
  }
  translate([-7, -15, -5]) cube([5, 10, 15]);
}
```

A simple "house" with a cube base, a rectangular open space for a door, a roof with a triangular cross section, and a chimney with a hexagonal cross section.

The initial cube creates the vertical walls of the house. The roof was then added as a cylinder that was rotated to have one end facing the front of the house, and translated to that the top of the initial cube runs exactly through its center. Setting $fn = 4 for this cylinder gives it a square profile, creating a 90-degree peak at the top of the roof. This shape also could have been produced using another cube, but it would have required a slightly more complicated rotation, and we would have needed a factor of the square root of 2 to calculate the correct dimensions.

The chimney is a simple cylinder protruding through the roof. This cylinder is actually a heptagonal (7-sided) prism, due to letting OpenSCAD automatically choose the number of sides for this scale. Finally, we used a union to join all of these shapes together, and a difference to subtract a smaller cube from this unioned shape to create a doorway into our house. If you would like to try a more-complex example, in our *Make: Geometry* book we gradually build up a complex castle in OpenSCAD. The photo that follows is the final state, from Chapter 13.

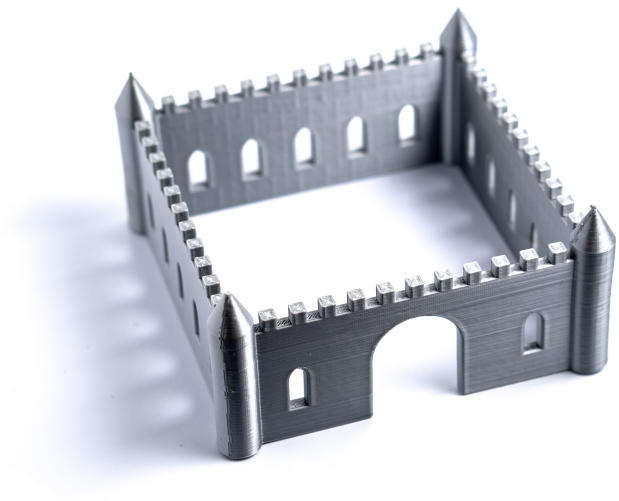

Figure 1: A square castle with turrets at each corner, a rounded main door, and Gothic arch windows all around. Crennelations are arranged all around the top of each wall.

Advanced Features: hull(), offset(), and minkowski() Modules

OpenSCAD's hull() module is used for creating a "convex hull" of one or more shapes. A convex hull is the smallest shape (2D or 3D, depending on the type of shapes inside) that surrounds a shape or shapes without any discontinuities between shapes, and with no concave (inward-facing) points. You can think of this like putting a layer of shrinkwrap around the shapes that wraps around them, as well as the space between them. For example, one quick way to create a shape with rounded corners in OpenSCAD is to place a circle at each corner, then wrap those circles in a hull() to fill in the space in between. This makes a (2D) square with rounded corners:

```
hull() {
  translate([0, 0, 0]) circle(5);
  translate([10, 0, 0]) circle(5);
  translate([0, 10, 0]) circle(5);
  translate([10, 10, 0]) circle(5);
}
```

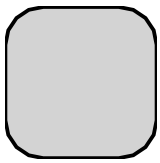

Figure 2: A square with rounded corners, file hull.svg

A square with rounded corners, file hull.svg

The hull() module generally processes very quickly in 2D, but may take a bit longer when working in 3D, especially using spheres, because they can have so many more facets for OpenSCAD to calculate.

To make a more complex 2D shape with rounded corners, especially if it has concave sections that you want to preserve, the offset() module may be even more useful. An offset lets you push all of the edges of a shape outward. When you push the edges outward, any convex (outward-pointing) corners between them will become rounded, with a radius equal to the distance they were pushed out. If you use a negative value, the corners will be pushed inward, and concave corners will be rounded. However, any sections of that shape with a width of twice that distance or thinner will be lost.

Just doing a single offset will change the dimensions of a shape. However, you can easily add round corners to a 2D shape without otherwise affecting its dimensions by doing a negative offset, followed by a positive offset of the same amount. Likewise, you can round the concave corners by doing the same thing in the opposite order. If you want to do both, you can do it with a series of three

offsets. Just make sure that the sum of all of the successive offsets equals zero to maintain your dimensions, and that there is no point along the way where the total offset is a negative value equal to half of your minimum width. This code makes a cross-shaped 2D object:

```
offset(5) offset(-15) offset(10) {
  square([20, 80], center = true);
  square([80, 20], center = true);
}
```

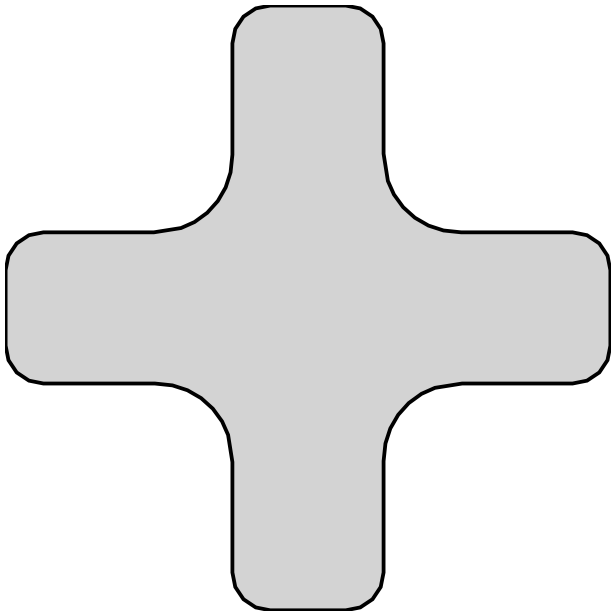

Figure 3: A cross shape with all its hard corners gently rounded

A cross shape with rounded edges, file offset.svg

A third way to round off corners in OpenSCAD is to use the minkowski() module. This is the simplest of the three methods to write, but the most computationally intensive to calculate. It works by taking one shape and "applying" it to another, such that you are essentially placing copies of one shape at each vertex of the other. While a minkowski() operation works in 3D, it can be extremely slow and memory-intensive (especially when using spheres with large facet counts, which is usually what you would want). Thus, people generally try to avoid using this operation in 3D. A 2D minkowski() operation requires much less processing, and usually will not require excessive amounts of processing time or memory. For example, this creates a 2D square with rounded corners:

```
minkowski() {
  circle(5);
```

```
  square(10);
}
```

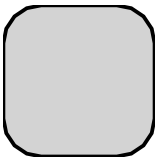

Figure 4: A square with rounded corners, file minkowski.svg

A square with rounded corners, file minkowski.svg

Teaching Tip: Creating in 3D by Starting in 2D

Many of OpenSCAD's operations are faster in 2D than in 3D. Working in 2D also means that you do not need to write a height value (either as a number or a variable name) over and over for each primitive when building up a shape in which they will all be the same height. You only need to use this value once, when you extrude a 2D shape into 3D.

A less obvious advantage comes when you want to center things. When designing for 3D printing, it is often advantageous to design an object with all of its parts sitting flat on the print bed. In the OpenSCAD house example, for instance, the walls were created from a cube with the center of its bottom face at the origin. To do this, a 20 mm cube was translated by -10 mm in the x and y directions, but zero in z.

```
translate([-10, -10, 0]) cube(20);
```

This could also have been done by centering the cube, but then it would also have been centered in Z, so it would still need to be translated up to achieve the same results.

```
translate([0, 0, 10]) cube(20, center = true);
```

In either case, you need to know the dimensions of the cube to know how far to translate it, and if you were doing something more complex, you would use a variable for this.

```
height = 20;
translate([0, 0, height / 2]) cube([10, 30, height], center = true);
```

Cylinders do not have this problem, because centering a cylinder only centers it in the z dimension. It is already centered in x and y. However, when you create a rectangular solid by using linear_extrude() on a square, you can separately control whether it is centered in the x, y plane and the z axis. To get a 20 mm cube with the center of its bottom face at the origin, all we need is:

```
linear_extrude(20) square(20);
```

With experience, it will become clearer when this technique makes sense. Take a look at some of the models in the repositories we describe in Section C for examples.

Learning More

Programming of any sort is not easy. OpenSCAD is a little quirky relative to some languages, but it is a good way of learning geometrical concepts through a text interface. With practice, it is possible to create very complex models. In Section C we give a general review of the models created for the books *Make: Geometry*, *Make: Trigonometry*, and *Make: Calculus* and our two books of science projects, and how to find the various models.

> **Note:** the OpenSCAD screenshot images above already have real descriptive alt text on the original site — carry that over as-is. The plain SVG diagrams (hull.svg, offset.svg, minkowski.svg) currently have empty alt text and would benefit from real descriptions when you migrate the actual image files.

Previous: Activity B.2: Moving from 2D to 3D · Download the whole thing · **Next:** Section C: 3D Printable STEM Models · **Back to:** Adventure Map